DAY 11 / 15 · PHASE 3 — ADVANCED

Gas Optimisation & Upgradeable Contracts

Sunday 25 October 2026 · Manish Khadka · Learning Blockchain series · code: github.com/Khadka1996/Learning-Blockchain/tree/main/day-11

1. Learning objectives

- Explain why **storage writes** dominate gas cost and name the real prices (20,000 / 2,900 / 2,100).
- Apply eight savings (packing, `immutable`, custom errors, `calldata`, cached reads, `unchecked`, one total write) and **prove** behaviour did not change with a differential fuzz.
- Explain a UUPS proxy: `delegatecall`, the ERC-1967 slot, `initialize`, locked implementations.
- Upgrade V1 → V2 without losing a balance, and recognise the classic reordered-storage disaster.

2. The concept, step by step

Part A — Gas is mostly about storage

Every step a contract takes costs **gas**. An addition costs 3. Writing to **storage** — the contract's permanent hard disk made of 32-byte "boxes" (slots) — is the expensive part. Think of a **post office with lockers**: opening a locker costs money whether it holds a letter or a parcel, so use fewer lockers.

Action	Gas
Empty box → filled	20,000
Change a box that already holds something	2,900
First touch of a box in a transaction (cold access)	+2,100
Add two numbers	3

Naive Member: 4 slots
bool · uint256 · uint256 · address

→

Packed Member: 1 slot
address 20 B + uint64 8 B + uint32 4 B = 32 B

Trick	Why it saves gas
Storage packing	One box instead of four; the box is already non-zero, so later writes cost 2,900 not 20,000
`immutable`	Value lives in the code: ~3 gas to read instead of 2,100

Custom errors	4-byte selector instead of a stored string; can carry data
`calldata`	Array read in place, no copy to memory
Cache length / values	Memory reads ~3 gas vs storage 100–2,100
One write per batch	Add up locally, write the total once
`unchecked { ++i; }`	Skip an overflow check that can never trigger

Warning: packing `points` into a `uint32` means it *must* be range-checked. Packing that silently wraps is a bug, not an optimisation.

Part B — Upgradeable contracts (UUPS proxy)

Deployed code can never change. A **proxy** is a shop front with a fixed address that holds the money and records; the **implementation** is the staff (code only). The proxy forwards every call with `delegatecall` — "run your code on *my* storage". Upgrading = pointing the proxy at new staff.



Term	Plain meaning
delegatecall	Borrow another contract's code, run it on your own storage
ERC-1967 slot	Fixed pseudo-random slot where the proxy keeps the implementation address, so it can't collide with variables
UUPS (ERC-1822)	Upgrade function lives in the implementation; cheaper proxy, but every version must keep it or the proxy is frozen
initialize()	Replaces the constructor (proxies never run one); runs once
_disableInitializers()	Locks the bare implementation so nobody can claim it

The classic mistake. V1 keeps `balanceOf` in slot 0 and `totalDeposits` in slot 1. A V2 that declares them in the other order compiles and upgrades happily — then every balance reads 0 while 5 ETH sit in the contract. Only *append* new variables.



3. Key code walkthrough

Packed struct and one-write batch (ClubOptimized)

```
struct Member { address referrer; uint64 joinedAt; uint32 points; } // 32 B = 1 slot
address public immutable owner;                // baked into code, ~3 gas to read

function addPointsBatch(address[] calldata list, uint256 amount) external {
    if (msg.sender != owner) revert NotOwner();
    uint256 len = list.length;                 // cache the length
    for (uint256 i; i < len; ) {
        Member storage m = members[list[i]];   // one storage pointer
        if (m.joinedAt == 0) revert NotMember(list[i]);
        uint256 room = type(uint32).max - uint256(m.points);   // QA E1
        if (amount > room) revert PointsOverflow(list[i]);
        m.points = uint32(uint256(m.points) + amount);
        unchecked { ++i; }                     // i < len: cannot overflow
    }
    totalPoints += len * amount;               // ONE write for the whole batch
}
```

The upgrade gate (PiggyBankV1)

```
constructor() { _disableInitializers(); }                // lock the bare implementation
function initialize(address o) public initializer { __Ownable_init(o); ... }
function _authorizeUpgrade(address) internal override onlyOwner {}  // the ONLY gate
```

Forget `onlyOwner` on `_authorizeUpgrade` and anyone can replace the code. V2 appends `maxPerUser` and `paused` and sets them in `initializeV2` (`reinitializer(2)`, owner only).

4. Results from the run

Action	Naive	Optimized	Saved	%
deploy	613,601	549,965	63,636	10.4%
join (no referrer)	96,580	67,939	28,641	29.7%
join (with referrer)	146,331	78,825	67,506	46.1%
addPointsBatch (10)	299,741	105,684	194,057	64.7%
addPointsBatch (50)	1,382,539	419,580	962,959	69.7%
addPointsBatch again (50)	527,539	419,580	107,959	20.5%
withdraw	30,843	28,743	2,100	6.8%

- Most of the 69.7% is one fact: naive `points` boxes start empty (20,000 gas), the packed box was filled at `join` (2,900).
- The "again" row (every box already non-zero in both) still saves 20.5% from the small tricks.
- `withdraw` saves exactly 2,100 gas: one cold read of `owner` removed.

- Upgrade demo: same proxy address, balances Alice 3 ETH / Bob 2 ETH kept, version 1.0.0 → 2.0.0; Eve's upgrade attempt reverted; the reordered V2 was refused by the storage-layout check.

5. QA & security findings

39 / 39 tests pass; differential fuzz (3 × 200 random programs, ~1,250 steps each) identical on both clubs; ESLint + Solhint clean; `npm audit` 0 vulnerabilities; Slither: nothing new in production code.

ID	Sev.	Finding	Fix
L1	Low	Single-step ownership: a typo could lose upgrade control forever	Ownable2Step
T1	Low	Storage-layout test passed for the wrong reason	Broken V2 given a proper initializer; assert the exact error
E1	Low	Huge reward gave raw Panic(0x11), not typed PointsOverflow	Compare by subtraction (~10 gas/member)
E5	Low	0-value deposit blamed on the cap after the cap was lowered	Check ZeroAmount first
P1–P5, E2–E4	Info	Init front-running, locked implementations, trapped funds, forced ETH, duplicate batch entries, uint32 boundary, partial batch	Verified safe; regression tests

Known issue K1: the owner can upgrade to *any* code — the trust assumption of every upgradeable contract. Production owner = multisig behind a timelock.

6. Common mistakes & interview questions

- Packing small types and forgetting the range check (silent wrap).
- Optimising without measuring; "clever" code that saves 20 gas but adds a bug.
- Reordering, removing or retyping state variables in an upgrade.
- Using a constructor in an upgradeable contract (it never runs on the proxy).
- Leaving the implementation uninitialised, or forgetting `onlyOwner` on `_authorizeUpgrade`.

Q1. Why is a storage write so expensive?
It changes the permanent state every node must keep; 20,000 gas for empty→non-zero, 2,900 for a change, +2,100 for a cold access (EIP-2929).

Q2. How does struct packing work and what is the risk?
Adjacent small fields share one 32-byte slot (address 20 + uint64 8 + uint32 4). Risk: narrow types overflow, so they need explicit range checks.

Q3. What is the difference between a transparent proxy and UUPS?
Transparent proxies hold the upgrade logic in the proxy (bigger, admin checks on every call); UUPS keeps it in the implementation (cheaper), but a version without the upgrade function bricks the proxy.

Q4. Why can't an upgradeable contract use a constructor?
The constructor runs on the implementation's own storage at deploy; the proxy's storage never sees it. Use `initialize` guarded by `initializer`, and lock the implementation.

Q5. What makes a V2 storage-layout compatible?
Every existing variable keeps its slot, name and type; new variables are only appended (or use ERC-7201 namespaced storage). Tools like OpenZeppelin's upgrades plugin check this.

7. Evening exercises

1. **Easy:** `slotsUsed(fields)` — naive member = 4 slots, packed = 1; field order matters.
2. **Medium:** predict the storage gas of rewarding N new members with the real prices; model says 76.0% vs measured 69.7% for N = 50.
3. **Challenge:** `checkUpgrade(old,new)` that flags reorder/removal/retype, confirmed against a real proxy's slot 1.

8. Cheat sheet

Gas: new box 20,000 · change 2,900 · cold +2,100 · add 3.
Savings: pack + range-check · immutable · custom errors · calldata · cache · one total write · unchecked loop counter.
Proxy: proxy = storage + ETH, implementation = code · delegatecall · ERC-1967 slot · initialize once · `_disableInitializers()` · `_authorizeUpgrade` + onlyOwner.
Upgrade rules: append only · never reorder/retype · run the layout check · owner = multisig + timelock.
Measure: same scenario, real receipts, differential fuzz for equal behaviour.

9. References

1. Solidity docs, Layout of state variables in storage — https://docs.soliditylang.org/en/v0.8.24/internals/layout_in_storage.html
2. EIP-2929: Gas cost increases for state access opcodes — https://eips.ethereum.org/EIPS/eip-2929
3. EIP-1967: Proxy storage slots — https://eips.ethereum.org/EIPS/eip-1967
4. EIP-1822: Universal Upgradeable Proxy Standard — https://eips.ethereum.org/EIPS/eip-1822
5. EIP-7201: Namespaced storage layout — https://eips.ethereum.org/EIPS/eip-7201
6. OpenZeppelin, Proxy upgrade pattern — https://docs.openzeppelin.com/upgrades-plugins/proxies
7. OpenZeppelin, Writing upgradeable contracts — https://docs.openzeppelin.com/upgrades-plugins/writing-upgradeable
8. Solidity blog, Custom errors — https://soliditylang.org/blog/2021/04/21/custom-errors/
9. evm.codes, opcode gas reference — https://www.evm.codes/
10. Antonopoulos & Wood, Mastering Ethereum — https://github.com/ethereumbook/ethereumbook